

Task 2

Define a state Σ that contains:

- Red Box Position
- Blue Box Position
- Turn (i.e. Red or Blue)

Define a directed, unweighted graph structure, in which each vertex is a possible game state, as defined above, and each edge is a possible move taken to traverse between game states. Define the source vertex s as the starting game state as described in the problem description, and the target vertex t as the ending game state. Run BFS from s , keeping track of distances, as seen in *Algorithm 53.5* from the textbook. When encountering line 6 of that algorithm, compute the neighbors dynamically from the grid value at the opposite token's position and the updated turn value (if red is taking the turn, then blue, and vice versa). Before running `union` for the new frontier, ensure positions are on the grid, and the two positions are not overlapping. After the conclusion of BFS, if t is in the domain of the visited table, return the distance associated with it. Otherwise t is not reachable.

The cost of canonical BFS is $O(|V| + |E|)$. We know $|V|$ represents the number of game states, which is $\# \text{red pos} \cdot \# \text{blue pos} \cdot \# \text{turns} = n^2 \cdot n^2 \cdot 2 \in O(n^4)$. Furthermore, We know from every state, in the worst case we have 4 possible moves to transition states, thus $|E| = 4 \cdot n^4 \in O(n^4)$.

Task 3

First, run a preprocessing step to compute the shortest path distances from s to every vertex, and likewise from t to every vertex. This can be done using two parallel BFS runs with distance tracking, as seen in *Algorithm 53.5*. Then, iterate through every possible combination of buildings u and v and count it as a road that doesn't reduce the shortest path distance if

1. Road not already constructed
2. $\delta(s, u) + 1 + \delta(v, t) \geq \delta(s, t)$
3. $\delta(s, v) + 1 + \delta(u, t) \geq \delta(s, t)$

Then, return the resulting count of roads.

Note that for any undirected graph, $\max(|E|) = \frac{n(n-1)}{2} \in O(n^2)$. The preprocessing step, computing the prefix and suffix distances has $O(|V| + |E|) \in O(n^2)$ work and span. The counting step iterates through every edge, so it also has $O(|E|) \in O(n^2)$ work and $O(1)$ span.

Task 4

First, create the graph structure as nodes as cities, edges as bus and helicopter routes, and the capital city as the source vertex. Then, run Dijkstra's algorithm from the capital city c (*Algorithm 56.2*) on the graph with both bus and helicopter routes as edges. In the resulting visited table X , you will have $\delta_{\text{all}}(c, V)$ for all V . Then, run Dijkstra's algorithm from the capital city c (*Algorithm 56.2*) on the graph with only bus routes as edges. In the resulting visited table X , you will have $\delta_{\text{bus}}(c, V)$ for all V . For each helicopter edge (c, U) with weight w , count it for removal if

1. $w > \delta_{\text{all}}(c, U)$
2. $\delta_{\text{bus}}(c, U) = \delta_{\text{all}}(c, U)$
3. If $\delta_{\text{bus}}(c, U) > \delta_{\text{all}}(c, U)$ & $w > \delta_{\text{all}}(c, U)$, then this edge is potentially needed. Among all helicopter routes to the same destination U , with the same optimal weight, keep one and count the rest for removal.

Return the end count of helicopter edges for removal.

The cost of the first Dijkstra's run is $O(m \log n + k \log n)$ and the second, $O(m \log n)$. The cost of checking each helicopter edge is $O(k)$, so the total cost of the solution is in $O(m \log n + k \log n)$.